OPTIMIZATION OF GARBAGE COLLECTION ROUTES IN BARANGAY VIRGEN DELAS FLORES

An Undergraduate Exposition Paper Presented to
Mr. Aarhus M. Dela Cruz
of the College of Science,
Bulacan State University,
City of Malolos, Bulacan

In Partial Fulfillment of the Requirements for
the Course Mathematical Modeling under
the Degree Bachelor of Science in Mathematics
with Specialization in Computer Science

By
COMPUTER SAIYANS
ORLANDA, GABRIELLE SEBASTIAN P.
MANALO, AIZLE
HERNANDEZ, MARY MAPLE T.
ROSAL, JAM RUSSEL M.
TOMAS, MARK RENIEL R.
TORIBIO, JOHN ARVIN R.

2nd Semester, AY 2025–2026

TABLE OF CONTENTS

Abstract **iv**

The Problem and Its Background **1**

Background of the Study 1

Statement of the Problem 2

Scope and Limitations 3

Significance of the Study 4

Graph-Theoretic Foundations of the Model **6**

Fundamentals of Graph Theory 6

Eulerian Graph 6

Chinese Postman Problem 7

Dijkstra's Algorithm 8

Mathematical Modeling Foundations of the Model **10**

Ant Colony Optimization 10

Methodology **13**

Description of the Real-Life System 13

Construction of the Graph Model 13

Modeling Assumptions and Constraints 15

Applied Graph-Theoretic Algorithms 15

Dijkstra's Algorithm 15

Results and Discussion **16**

Introduction to using Ant Colony Optimization as a Metaheuristic Method . . 16

Application of Dijkstra's Algorithm for Ant Colony Optimization 16

Odd-Degree Vertex Pairing using Ant Colony Optimization 19

Ant Colony Optimization Generation Output 21

Traversing the Eulerian Map 22

Summary and Conclusion **25**

Summary of Findings 25

Conclusions 25

References **27**

Appendix A: Vertex and Edge List **28**

Appendix B: SageMath/Python Codes **29**

ABSTRACT

Garbage Collection is an essential foundation for environmental and neighborhood cleanliness. It occurs as Garbage Collection Vehicles traverse through the entire neighborhood with its duty to collect and dispose of waste properly. The study aims to create an optimal gabage collection route for Barangay Virgen Delas Flores by using fundamental concepts in Graph Theory such as the Eulerian Graphs by attibuting the problem as a Chinese Postman Problem. Through the use of a metaheuristic approach by using a Mathematical Model, it was possible to determine a final estimated efficient route by using the Ant Colony Optmization Metaheuristic as well as using Dijkstra's Algorithm to determine the shortest path to the solution, with the total route having a final value of 5440.68 meters. The study concludes that applying these mathematical models provides a significant advantage for waste management personnel and local government units by enhancing operational productivity and reducing environmental impact.

Keywords: Chinese Postman Problem, Ant Colony Optimization, Eulerian Graph, Garbage Collection Routing

THE PROBLEM AND ITS BACKGROUND

This chapter outlines the primary problem addressed by the study, focusing on inefficiencies in garbage collection routes in Barangay Virgen Delas Flores. It highlights the challenges of manual route planning, which lead to increased time, cost, and resource use. It also identifies the need for systematic, data-driven approaches in waste management. Ultimately, this chapter sets the stage for the proposed use of graph theory and mathematical modeling to improve route efficiency and overall waste collection.

Background of the Study

Solid waste management is important in ensuring the maintenance of public health, environmental protection, and the sustainability of communities. Waste management systems are not only indicative of the technical capabilities of a system, but also mirror societal structures, governance, and societal behaviors, thereby underlining their societal significance (Reno, 2015). Waste management is constantly becoming a problem in both urban and semi-urban areas, thereby making garbage collection services a necessity in both areas.

In areas such as Barangay Virgen Delas Flores in Baliwag City, garbage collection activities involve the traversal of connected streets and areas within a predetermined period, thereby requiring the development of effective garbage routes in order to ensure the effective management of waste in the area. Effective management of routes is indicative of effective environmental management, which is characterized by the conservation of resources and the improvement of system operations (Wang et al., 2018).

The purpose of this study is to create an optimized garbage collection route for Barangay Virgen Delas Flores through the systematic collection of data regarding the barangay's road network and the application of optimization methods to determine the

routes with the least total distance traveled and the routes that cover all necessary paths. This objective is formally modeled as an optimization problem on a weighted graph $G = (V, E)$, where each edge has an associated cost. The study seeks to minimize the total traversal cost while ensuring complete edge coverage and satisfying the parity conditions required for a valid traversal. The complete mathematical formulation is presented in the succeeding sections.

The application of a systematic approach in determining the garbage collection route for Barangay Virgen Delas Flores demonstrates how optimization methods can be utilized in the decision-making process in waste management systems in the Philippines. The results of this study can possibly serve as a guide for barangays in optimizing their garbage collection routes to make it more efficient in the Philippines.

Statement of the Problem

Garbage collection is an essential part of community systems that plays a vital role in preventing the spread of diseases, pest infestations, and environmental contamination. However, the current garbage collection system in Barangay Virgen Delos Flores shows inefficiencies in terms of time and resource. It is visible by observing the repeated and non-optimized routes taken by garbage trucks.

These inefficiencies emphasizes the need for more systematic and optimized approach to route planning. Mathematical and logical methods can provide potential solutions to improve the efficiency the is kind of systems. In particular, the application of graph theory and optimization techniques can provide a structured way of analyzing and improving garbage collection routes. The study seeks to answer the question, "How mathematical concepts, particularly graph theory and mathematical modeling can be utilized to enhance garbage collection route efficiency?".

To further emphasize its relevance, this research seeks to answer the following questions:

1. How can the road network be represented as a weighted graph?
2. What methods can be used to improve route efficiency and minimize total distance traveled?
3. How can optimization techniques be applied to solve routing problems?
4. What is the effect of applying optimization methods on overall route efficiency?

Scope and Limitations

The scope of this study encompasses the following parameters:

1. **Geographic Coverage:** Limited to garbage collection routes within Barangay Virgen Delas Flores.
2. **Network Representation:** The road network is modeled as a weighted graph where intersections and endpoints are represented as vertices, and roads are represented as edges with corresponding distance values.
3. **Shortest Path Computation: Dijkstra's Algorithm** is utilized to determine the shortest paths between locations.
4. **Route Coverage:** The **Chinese Postman Problem (CPP)** is applied to ensure that all roads are included in the collection route.
5. **Optimization Technique: Ant Colony Optimization (ACO)** is used to identify optimal pairings of odd-degree vertices to improve route efficiency.
6. **Performance Metric:** The efficiency of the route is evaluated primarily based on the total distance traveled.
7. **Expected Output:** The study aims to generate a route that covers all required paths while minimizing unnecessary travel.

The study acknowledges the following limitations:

1. **Static Conditions:** All roads are assumed to be available and usable at any time during the garbage collection process.
2. **External Factors:** Traffic conditions, road closures, and weather conditions are not considered in the analysis.

3. **Operational Constraints:** Factors such as fuel consumption, vehicle capacity, manpower, and scheduling are excluded from the study.
4. **Evaluation Basis:** Route efficiency is assessed solely based on distance, without considering time or cost-related factors.

Significance of the Study

The study aims to improve the efficiency of garbage collection in Barangay Virgen Delas Flores by optimizing collection routes using graph theory and mathematical modeling. Through the use of Eulerian Graph, Chinese Postman Problem, Dijkstra's Algorithm, and Ant Colony Optimization. Seeking for minimization of travel time, reduce fuel consumption, and enhance overall waste management operations.

Residents. As the primary beneficiaries of the proposed garbage collection system. This reduces the accumulation of garbage in the community, leading to cleaner surroundings and lower health risks.

Waste Collection Personnel. The optimized routes help garbage collectors perform their tasks more efficiently by eliminating unnecessary or repetitive paths. It allows for reducing physical strain, saves time, and improves productivity.

Barangay Officials. The study provides barangay officials with a data-driven basis for improving their waste management system.

Local Government Units (LGU). The findings of this study can serve as a model for other barangays and municipalities aiming to improve their solid waste management systems.

Future Researchers. This study contributes to the application of graph theory, mathematical modeling, and optimization algorithms in real-world problems. It serves as a reference for future studies related to logistics, urban planning, and environmental

management, and may inspire further improvements or innovations in route optimization methods.

GRAPH-THEORETIC FOUNDATIONS OF THE MODEL

This section introduces the fundamental concepts of Graph Theory that are used to formulate and analyze the garbage collection routing problem.

Fundamentals of Graph Theory

Graph Theory is a branch of mathematics in which it focuses on the study of graphs, networks of points that are connected by lines. It is applied in multiple fields such as computer science, engineering, social network structures, and more real world applications that help solve real world issues. Graphs in Graph Theory are defined as the set of vertices and edges which connect vertices (Carlson, 2026).

A graph is formally defined as an ordered pair:

$$G = (V, E)$$

where V is a set of **vertices** (or nodes) and E is a set of **edges** (or links). Each $e \in E$ is assigned a weight such that it usually represents a quantitative measure such as distance, time, or capacity. A **Trail** is an open walk where no edge is repeated If a path begins and ends at the same vertex, it is called a **Cycle**.

One way to represent a weighted graph is through a **Weighted Adjacency Matrix**. It is a $|V| \times |V|$ matrix, where $|V|$ is the number of vertices in the graph, where each entry A_{ij} stores the weight of the edge between vertex i and vertex j.

Eulerian Graph

In a graph, the **degree of a vertex** is the number of edges connected or incident to that vertex, it is denoted as:

$$deg(v)$$

Vertices in a graph can be classified based on their degree, either even-degree vertices or odd-degree vertices (Carlson, 2026).

An Eulerian graph is a connected graph that contains a closed trail which passes through every edge exactly once and returns to the starting vertex. For a graph to be Eulerian, every vertex in the graph must have an even degree. Otherwise, if a graph contains a trail that visits every edge exactly once but does not return to the starting vertex it is considered semi-Eulerian (Calverd, 2025). In this study's case, the traversal would be depicted as a trail such that all edges are visited once.

Chinese Postman Problem

The Chinese Postman Problem (CPP) is a problem in graph theory that aims to find the shortest route that passes through every edge of a graph at least once and returns to the starting point. If all vertices in a connected graph have even degree, an Eulerian exists and each edge can be traversed exactly once then it can be used as the optimal route since each edge is traversed exactly once (Sgarro & Grilli, 2023).

However, if some vertices have an odd degree, an Eulerian cannot be formed. To solve this, the odd-degree vertices are paired and additional edges are added between them. These added edges represent routes that must be repeated. In weighted graphs, the goal is to choose the pairings that produce the smallest additional distance. After adding these edges, the graph becomes Eulerian and it can be used as the optimal route.

Let $G = (V, E)$ be a connected undirected graph where each edge $e \in E$ has a nonnegative traversal cost c_e.

Let $s \in V$ denote the designated entry vertex of the truck into the barangay, and $t \in V$ denote the designated exit vertex, where $s \neq t$. In this study, the entry and exit vertices are predetermined based on the nearest accessible boundary points of the barangay. The pairing process is then applied only to the remaining odd-degree vertices to satisfy the semi-Eulerian condition.

Decision Variable:

$$x_e \in \mathbb{Z}_{\geq 0} \quad \forall e \in E$$

where x_e represents the number of times edge e is traversed.

Mathematical Formulation:

$$\begin{aligned}
\min \quad & \sum_{e \in E} c_e x_e & \\
\text{s.t.} \quad & x_e \geq 1 & \forall e \in E \\
& \sum_{e \in \delta(v)} x_e \equiv \begin{cases} 1 \pmod 2, & \text{if } v = s \text{ or } v = t, \\ 0 \pmod 2, & \text{otherwise} \end{cases} & \forall v \in V \\
& x_e \in \mathbb{Z}_{\geq 0} & \forall e \in E
\end{aligned}$$

where $\delta(v)$ denotes the set of edges incident to vertex v.

The parity constraint ensures that exactly two vertices that correspond to the designated entry and exit points of the truck to have odd degree, while all other vertices have even degree. This solidifies the formulation of a semi-Eulerian trail that traverses every edge at least once that begins at s and terminates at t. It is consistent with the operational scenario where garbage collection starts and ends outside the scoped area.

Dijkstra's Algorithm

Dijkstra's Algorithm is a well-known algorithm used to determine the shortest path between vertices in a weighted graph. It was developed by Edsger W. Dijkstra and is commonly used in network routing, transportation systems, and navigation applications.

The algorithm starts from a selected source vertex and explores its neighboring vertices while maintaining the shortest known distance from the source. At each step, the

vertex with the smallest tentative distance is selected, and the distances of its adjacent vertices are updated. This process continues until the shortest paths to all vertices are determined (Navone, 2020).

The steps of Dijkstra's Algorithm are summarized as follows:

1. Assign a distance value of 0 to the starting vertex and ∞ to all other vertices.
2. Mark all vertices as unvisited and set the starting vertex as the current vertex.
3. For the current vertex, check all its adjacent vertices and compute their distances.
4. If the computed distance is smaller than the current value, update it.
5. After checking all adjacent vertices, mark the current vertex as visited.
6. Choose the unvisited vertex with the smallest distance as the new current vertex.
7. Repeat the process until all shortest paths are found.

To formally describe the algorithm. Let $G = (V, E, w)$ be a weighted graph with vertex set V, edge set E, and nonnegative edge weight function $w : E \to \mathbb{R}_{\geq 0}$. Given a source vertex $s \in V$, Dijkstra's algorithm computes the *shortest path distances*

$$d(s, v) = \min_{p \in \mathcal{P}(s,v)} \sum_{e \in p} c(e)$$

for all $v \in V$, where $\mathcal{P}(s, v)$ denotes the set of all paths from s to v, by iteratively selecting the unvisited vertex with minimum tentative distance and updating neighboring distances. The algorithm assumes all edge weights satisfy $c(e) \geq 0$ (Black, 2024).

MATHEMATICAL MODELING FOUNDATIONS OF THE MODEL

This section presents the fundamental concepts of the Mathematical Model that will be used to formulate and analyze the garbage collection routing problem.

Ant Colony Optimization

Ant Colony Optimization (ACO) is a nature-inspired optimization algorithm based on the behavior of real ants when searching for food. In nature, ants are able to find the shortest path between their colony and a food source without any central control. They achieve this by leaving chemical trails called pheromones along the paths they travel. Over time, shorter paths accumulate stronger pheromone trails because they are traveled more frequently. As a result, other ants are more likely to follow those paths, which eventually leads the colony to identify the shortest route.

In the Ant Colony Optimization algorithm, artificial ants simulate this behavior when exploring possible solutions to a problem. Each ant selects the next node based on a probabilistic transition rule that considers both the pheromone level and a heuristic value related to the path cost. The probability of selecting the next node is defined as:

$$P_{ij} = \frac{(\tau_{ij}^{\alpha})(\eta_{ij}^{\beta})}{\sum(\tau_{ij}^{\alpha})(\eta_{ij}^{\beta})} \tag{1}$$

where τ_{ij} is the amount of pheromone deposited on the edge ij, η_{ij} is the travel cost (heuristic) of the edge ij ($\eta_{ij} = \frac{1}{c_e}$), α controls the influence of the pheromone value, and β controls the influence of the heuristic value (Roopawalla, 2024).

The Distance represents how good a path looks based on its distance where:

$$\eta_{ij} = \frac{1}{d_{ij}} \tag{2}$$

where d_{ij} is the shortest distance between nodes i and j.

After all ants complete their paths, pheromones are updated:

$$\tau_{ij} = (1-\rho)\tau_{ij} + \sum_{k=1}^{m} \Delta\tau_{ij}^{k} \tag{3}$$

where ρ is the coefficient of evaporation and m is the number of ants.

The pheromone contribution of each ant is given by

$$\Delta\tau_{ij} = \begin{cases} \frac{Q}{L} & \text{if the path is used} \\ 0 & \text{otherwise} \end{cases} \tag{4}$$

where Q is the constant value and L is the total length of the path.

Through repeated iterations, the pheromone trails guide ants toward the optimal or near-optimal solution (Sgarro & Grilli, 2023). This Metaheuristic approach will serve as the Mathematical Modeling part of the study.

The steps of ACO Algorithm are summarized as follows:

1. Initialize pheromone values and parameters for all possible paths.
2. Compute the transition probabilities for ants based on pheromone levels and heuristic distance.
3. Construct solutions as each ant builds a complete path using these probabilities.
4. Evaluate the quality of each completed solution based on the total distance.
5. Update pheromones by evaporating existing trails and depositing new ones on the best-performing paths.
6. Repeat the process until a near-optimal solution is obtained.

In the Chinese Postman Problem, transforming a non-Eulerian graph into an Eulerian graph requires pairing all odd-degree vertices. When the number of odd vertices is small, these pairings can be determined through direct comparison of

distances. However, as the number of odd-degree increases, the number of possible pairings grows rapidly, making direct comparison impractical.

This pairing can be viewed as a combinatorial optimization problem, where the objective is to select pairs of vertices such that the total additional distance into the graph is minimized (Siddique & Adeli, 2017). Ant Colony Optimization (ACO) is well suited for such problems because it efficiently explores large search spaces through probabilistic solution and pheromone updating.

Studies such as those by Sgarro and Grilli (2023), show that ACO can effectively determine near-optimal pairings in Chinese Postman variations. In this study, ACO is used to identify efficient pairings of odd-degree vertices based on shortest path distances, enabling the graph to become Eulerian with minimal added cost.

METHODOLOGY

The following section shows the steps on how the Chinese Postman Problem will be solved through the use of Dijkstra's Algorithm and Ant Colony Optimization.

Description of the Real-Life System

The Barangay Virgen Delas Flores is located in the Municipality of Baliuag, Bulacan, Philippines. The garbage collection schedule and routes span separately within a whole week starting from Monday to Friday.

Figure 1

Barangay Virgen Delas Flores Situated Map

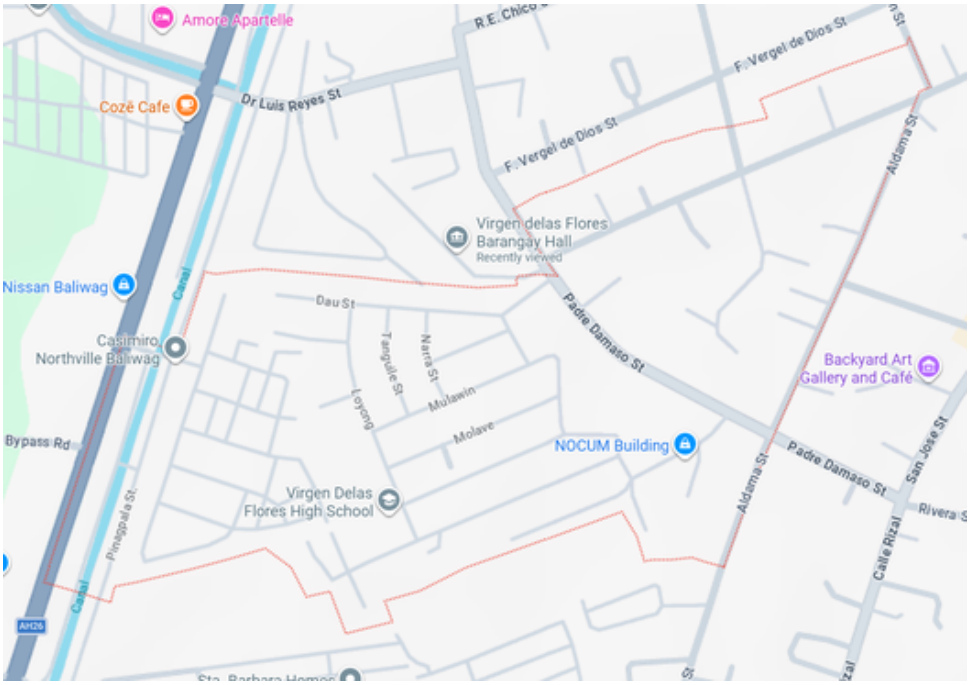


Figure 1 map structure includes several roads connected by intersections joining two or more roads incident to the intersection. It also includes cul-de-sacs or dead ends.

Construction of the Graph Model

The scope of the study includes all reachable edges within the barangay excluding those edges that cant be traversed by a Garbage Truck via on-site observation as well as

scheduled routing.

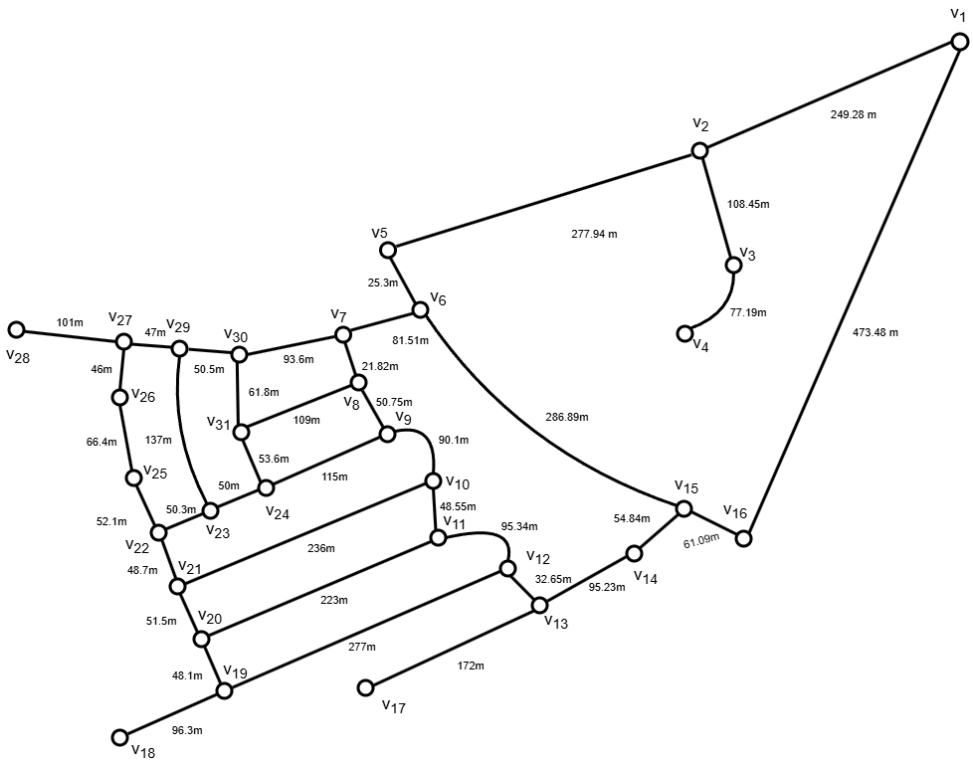
Let $G = (V, E, w_G)$ be the graph of the situated map of Barangay Virgen Delas Flores where:

- V are the set of vertices which represents intersections and cul-de-sac (Dead End).
- E are the set of roads incident to existing intersections.
- w_G are the constant distance between two nodes measured via Google Maps.

The following shows the converted Situated Map into an Undirected Graph of the Barangay.

Figure 2

Converted Barangay Situated Map into a Undirected Graph



Then:

- $V = \{v_1, v_2, v_3, v_4, \ldots, v_{30}, v_{31}\}$
- $E = \{\{v_1, v_2\}, \{v_2, v_3\}, \{v_3, v_4\}, \{v_2, v_5\}, \ldots, \{v_{30}, v_{31}\}, \{v_{31}, v_{24}\}\}$

Figure 2 shows the situated network map of Barangay Virgen Delas Flores

presenting all the intersections, road networks, and distance inbetween nodes.

Modeling Assumptions and Constraints

To ensure mathematical consistency of the proposed routing model, the following assumptions are established:

1. The road network of Barangay Virgen Delas Flores is connected and can be accurately represented as a weighted undirected graph $G = (V, E)$.
2. All roads are bidirectional and fully accessible to the garbage truck during the collection period.
3. Only one garbage truck operates within the barangay during a collection tour.
4. The truck has sufficient capacity to complete the collection without requiring intermediate unloading.
5. During the execution of the route, there are no unexpected events, such as road closures, construction, or vehicle breakdowns.

From the overall map observation, there are a total of 31 vertices present where the ratio of even to odd edges incident to the vertices are observed to be 7 to 24. The total number of edges observed are 38.

Applied Graph-Theoretic Algorithms

Dijkstra's Algorithm

Dijkstra's Algorithm is depicted as finding the shortest path between two vertices. This algorithm will be used for determining the vertex pairings in the Ant Colony Optmization as it is the most appropriate shortest paths algorithms to determine the algorithm's η_{ij} or Heuristic values.

RESULTS AND DISCUSSION

This sections includes the Results and Discussions of the problem solution. This includes the step-by-step explanation on how the algorithms are applied to find a possible optimal solution.

Introduction to using Ant Colony Optimization as a Metaheuristic Method

To solve the problem, the first step is to determine the number of odd-degree vertices in the graph.

Let O represent the set of odd-degree vertices present in graph G.

$$O = \{v_2, v_4, v_6, v_7, v_8, v_9, v_{10}, v_{11}, v_{12}, v_{13}, v_{15}, v_{17}, v_{18}, v_{19}, v_{20}, v_{21}, v_{22}, v_{23}, v_{24}, v_{27}, v_{28}, v_{29}, v_{30}, v_{31}\}$$

This gives the size of the odd-degree vertices in G as $|O| = 24$ which gives 12 pairs of odd-degree vertices. Because the size of the odd-degree vertices is 24, for regular odd-degree vertex pairings using weighted comparisons for the most optimal vertex pairings, there would be approximately $316,234,143,225$ number of distinct pairings which would be computationally exhaustive. Hence why one of the possible solutions is to use a Mathematical Modeling Metaheuristics approach by using the Ant Colony Optimization.

Application of Dijkstra's Algorithm for Ant Colony Optimization

To transform the graph into an Eulerian graph, it is necessary to determine the shortest path distances between all pairs of odd-degree vertices. These distances represent the additional cost when edges are duplicated during the pairing process.

To obtain these values, Dijkstra's Algorithm is applied to the weighted adjacency matrix shown in Table 1. For each odd vertex $u \in O$, the algorithm is used to determine the shortest path distance to every other vertex $v \in O$, where $u \neq v$. These shortest path distances represent the minimum cost required when duplicating edges between pairs of

Table 1

Weighted Adjacency Matrix of the Road Network of Barangay Virgen Delas Flores

	v_1	v_2	v_3	v_4	v_5	v_6	v_7	v_8	v_9	v_{10}	v_{11}	v_{12}	v_{13}	v_{14}	v_{15}	v_{16}	v_{17}	v_{18}	v_{19}	v_{20}	v_{21}	v_{22}	v_{23}	v_{24}	v_{25}	v_{26}	v_{27}	v_{28}	v_{29}	v_{30}	v_{31}
v_1	0	249.28	0	0	0	0	0	0	0	0	0	0	0	0	0	473.48	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
v_2	249.28	0	108.45	0	277.94	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
v_3	0	108.45	0	77.19	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
v_4	0	0	77.19	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
v_5	0	277.94	0	0	0	25.3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
v_6	0	0	0	0	25.3	0	81.51	0	0	0	0	0	0	0	286.89	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
v_7	0	0	0	0	0	81.51	0	21.82	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	93.6	0
v_8	0	0	0	0	0	0	21.82	0	50.75	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	109
v_9	0	0	0	0	0	0	0	50.75	0	90.1	0	0	0	0	0	0	0	0	0	0	0	0	0	115	0	0	0	0	0	0	0
v_{10}	0	0	0	0	0	0	0	0	90.1	0	48.55	0	0	0	0	0	0	0	0	0	236	0	0	0	0	0	0	0	0	0	0
v_{11}	0	0	0	0	0	0	0	0	0	48.55	0	95.34	0	0	0	0	0	0	0	223	0	0	0	0	0	0	0	0	0	0	0
v_{12}	0	0	0	0	0	0	0	0	0	0	95.34	0	32.65	0	0	0	0	0	277	0	0	0	0	0	0	0	0	0	0	0	0
v_{13}	0	0	0	0	0	0	0	0	0	0	0	32.65	0	95.23	0	0	17.2	0	0	0	0	0	0	0	0	0	0	0	0	0	0
v_{14}	0	0	0	0	0	0	0	0	0	0	0	0	95.23	0	54.84	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
v_{15}	0	0	0	0	0	286.89	0	0	0	0	0	0	0	54.84	0	61.09	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
v_{16}	473.48	0	0	0	0	0	0	0	0	0	0	0	0	0	61.09	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
v_{17}	0	0	0	0	0	0	0	0	0	0	0	0	17.2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
v_{18}	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	96.3	0	0	0	0	0	0	0	0	0	0	0	0
v_{19}	0	0	0	0	0	0	0	0	0	0	0	277	0	0	0	0	0	96.3	0	48.1	0	0	0	0	0	0	0	0	0	0	0
v_{20}	0	0	0	0	0	0	0	0	0	0	223	0	0	0	0	0	0	0	48.1	0	51.5	0	0	0	0	0	0	0	0	0	0
v_{21}	0	0	0	0	0	0	0	0	0	236	0	0	0	0	0	0	0	0	0	51.5	0	48.7	0	0	0	0	0	0	0	0	0
v_{22}	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	48.7	0	50.3	0	52.1	0	0	0	0	0	0
v_{23}	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	50.3	0	50	0	0	0	0	137	0	0
v_{24}	0	0	0	0	0	0	0	0	115	0	0	0	0	0	0	0	0	0	0	0	0	0	50	0	0	0	0	0	0	0	53.6
v_{25}	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	52.1	0	0	0	66.4	0	0	0	0	0
v_{26}	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	66.4	0	46	0	0	0	0
v_{27}	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	46	0	101	47	0	0
v_{28}	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	101	0	0	0	0
v_{29}	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	137	0	0	0	47	0	0	50.5	0
v_{30}	0	0	0	0	0	0	93.6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	50.5	0	61.8
v_{31}	0	0	0	0	0	0	0	109	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	53.6	0	0	0	0	0	61.8	0

odd vertices.

To illustrate the process, consider the computation of the shortest path from vertex v_2 to vertex v_4, as shown in Table 2.

Table 2

Dijkstra Iterations from v_2 to v_4

Vertex	v_2	v_1	v_3	v_5	v_4
v_2	$\boxed{0_{v_2}}$	249.28_{v_2}	108.45_{v_2}	277.94_{v_2}	∞
v_3		249.28_{v_2}	$\boxed{108.45_{v_2}}$	277.94_{v_2}	185.64_{v_3}
v_4		249.28_{v_2}		277.94_{v_2}	$\boxed{185.64_{v_3}}$
v_1		$\boxed{249.28_{v_2}}$		277.94_{v_2}	
v_5				$\boxed{277.94_{v_2}}$	

The computation starts at vertex v_2, where its distance is set to 0 and ∞ to all other vertices. The adjacent vertices of v_2 are then evaluated, and their temporary distances are updated.

Among these vertices, v_3 has the smallest temporary distance, so it is selected as the next vertex and v_2 is marked as visited. The distances of its adjacent vertices are then updated, resulting in the shortest path to v_4 from v_3 with a total distance of 185.64.

This process continues by repeatedly selecting the unvisited vertex with the

smallest temporary distance and updating its neighbors until the shortest path is determined.

- The first row shows the initial computation of neighbor vertices of v_2 and indicates the first recognized shortest path.
- The second row shows the shortest distance of v_2 to v_4 traversing through v_3 and recognizes v_3 as the next shortest path.
- The third row shows the goal of the Dijkstra Algorithm which is the recognized shortest path of v_2 and v_4.
- The rest of the rows shows the shortest path of the v_2 to the other neighbor vertices.

This process is applied to all odd-degree vertices in the graph to compute the shortest path distances between every pair of vertices in O. The computed values are then used to construct a reduced weighted adjacency matrix, where the rows and columns represent as the odd-degree vertices, and each entry corresponds to the shortest path distance between two vertices, as shown in Table 3.

Table 3

Shortest Path between Odd-Degree Vertices

	v_2	v_4	v_6	v_7	v_8	v_9	v_{10}	v_{11}	v_{12}	v_{13}	v_{15}	v_{17}	v_{18}	v_{19}	v_{20}	v_{21}	v_{22}	v_{23}	v_{24}	v_{27}	v_{28}	v_{29}	v_{30}	v_{31}
v_2	0	185.64	303.24	384.75	406.57	457.32	547.42	595.97	691.31	723.96	590.13	741.16	914.07	817.77	769.67	718.17	669.47	619.17	569.17	575.85	676.85	528.85	478.35	515.57
v_4	185.64	0	488.88	570.39	592.21	642.96	733.06	781.61	876.95	909.6	775.77	926.8	1099.71	1003.41	955.31	903.81	855.11	804.81	754.81	761.49	862.49	714.49	663.99	701.21
v_6	303.24	488.88	0	81.51	103.33	154.08	244.18	292.73	388.07	420.72	286.89	437.92	610.83	514.53	466.43	414.93	366.23	315.93	265.93	272.61	373.61	225.61	175.11	212.33
v_7	384.75	570.39	81.51	0	21.82	72.57	162.67	211.22	306.56	339.21	368.4	356.41	529.32	433.02	384.92	333.42	284.72	234.42	184.42	191.1	292.1	144.1	93.6	130.82
v_8	406.57	592.21	103.33	21.82	0	50.75	140.85	189.4	284.74	317.39	390.22	334.59	507.5	411.2	363.1	311.6	262.9	212.6	162.6	212.92	313.92	165.92	115.42	109
v_9	457.32	642.96	154.08	72.57	50.75	0	90.1	138.65	233.99	266.64	416.71	283.84	459.9	363.6	315.5	264	215.3	165	115	263.67	364.67	216.67	166.17	159.75
v_{10}	547.42	733.06	244.18	162.67	140.85	90.1	0	48.55	143.89	176.54	326.61	193.74	415.95	319.65	271.55	236	284.7	255.1	205.1	353.77	454.77	306.77	256.27	249.85
v_{11}	595.97	781.61	292.73	211.22	189.4	138.65	48.55	0	95.34	127.99	278.06	145.19	367.4	271.1	223	274.5	323.2	303.65	253.65	402.32	503.32	355.32	304.82	298.4
v_{12}	691.31	876.95	388.07	306.56	284.74	233.99	143.89	95.34	0	32.65	182.72	49.85	373.3	277	318.34	369.84	418.54	398.99	348.99	497.66	598.66	450.66	400.16	393.74
v_{13}	723.96	909.6	420.72	339.21	317.39	266.64	176.54	127.99	32.65	0	150.07	17.2	405.95	309.65	350.99	402.49	451.19	431.64	381.64	530.31	631.31	483.31	432.81	426.39
v_{15}	590.13	775.77	286.89	368.4	390.22	416.71	326.61	278.06	182.72	150.07	0	167.27	556.02	459.72	501.06	552.56	601.26	581.71	531.71	559.5	660.5	512.5	462	499.22
v_{17}	741.16	926.8	437.92	356.41	334.59	283.84	193.74	145.19	49.85	17.2	167.27	0	423.15	326.85	368.19	419.69	468.39	448.84	398.84	547.51	648.51	500.51	450.01	443.59
v_{18}	914.07	1099.71	610.83	529.32	507.5	459.9	415.95	367.4	373.3	405.95	556.02	423.15	0	96.3	144.4	195.9	244.6	294.9	344.9	409.1	510.1	431.9	460.3	398.5
v_{19}	817.77	1003.41	514.53	433.02	411.2	363.6	319.65	271.1	277	309.65	459.72	326.85	96.3	0	48.1	99.6	148.3	198.6	248.6	312.8	413.8	335.6	364	302.2
v_{20}	769.67	955.31	466.43	384.92	363.1	315.5	271.55	223	318.34	350.99	501.06	368.19	144.4	48.1	0	51.5	100.2	150.5	200.5	264.7	365.7	287.5	315.9	254.1
v_{21}	718.17	903.81	414.93	333.42	311.6	264	236	274.5	369.84	402.49	552.56	419.69	195.9	99.6	51.5	0	48.7	99	149	213.2	314.2	236	264.4	202.6
v_{22}	669.47	855.11	366.23	284.72	262.9	215.3	284.7	323.2	418.54	451.19	601.26	468.39	244.6	148.3	100.2	48.7	0	50.3	100.3	164.5	265.5	187.3	215.7	153.9
v_{23}	619.17	804.81	315.93	234.42	212.6	165	255.1	303.65	398.99	431.64	581.71	448.84	294.9	198.6	150.5	99	50.3	0	50	184	285	137	165.4	103.6
v_{24}	569.17	754.81	265.93	184.42	162.6	115	205.1	253.65	348.99	381.64	531.71	398.84	344.9	248.6	200.5	149	100.3	50	0	212.9	313.9	165.9	115.4	53.6
v_{27}	575.85	761.49	272.61	191.1	212.92	263.67	353.77	402.32	497.66	530.31	559.5	547.51	409.1	312.8	264.7	213.2	164.5	184	212.9	0	101	47	97.5	159.3
v_{28}	676.85	862.49	373.61	292.1	313.92	364.67	454.77	503.32	598.66	631.31	660.5	648.51	510.1	413.8	365.7	314.2	265.5	285	313.9	101	0	148	198.5	260.3
v_{29}	528.85	714.49	225.61	144.1	165.92	216.67	306.77	355.32	450.66	483.31	512.5	500.51	431.9	335.6	287.5	236	187.3	137	165.9	47	148	0	50.5	112.3
v_{30}	478.35	663.99	175.11	93.6	115.42	166.17	256.27	304.82	400.16	432.81	462	450.01	460.3	364	315.9	264.4	215.7	165.4	115.4	97.5	198.5	50.5	0	61.8
v_{31}	515.57	701.21	212.33	130.82	109	159.75	249.85	298.4	393.74	426.39	499.22	443.59	398.5	302.2	254.1	202.6	153.9	103.6	53.6	159.3	260.3	112.3	61.8	0

This reduced matrix serves as the input for the Ant Colony Optimization (ACO) algorithm, where these distances are used to evaluate and determine the optimal pairing of odd-degree vertices.

Odd-Degree Vertex Pairing using Ant Colony Optimization

After constructing the reduced weighted adjacency matrix of odd-degree vertices, the next step is to determine an optimal pairing of these vertices.

The Ant Colony Optimization algorithm works by letting ants start at each odd-degree vertex where Equation 1 will represent as the probability that the vertex pairings will be picked. Initializing the values where $\alpha = 1$, $\beta = 3$, and $\rho = 0.5$

Starting at vertex v_2, the first step is to determine the η_{ij}, where i represents the initial vertex v_2, and j represents the vertex to be paired with v_2. Using the shortest path distances obtained from Dijkstra's Algorithm and applying Equation 2, the values are as follows:

- $\eta_{v_2v_4} = \frac{1}{185.64} \approx 5.386 \times 10^{-3}$
- $\eta_{v_2v_6} = \frac{1}{303.24} \approx 3.297 \times 10^{-3}$
- $\eta_{v_2v_7} = \frac{1}{384.75} \approx 2.599 \times 10^{-3}$
- $\eta_{v_2v_8} = \frac{1}{406.57} \approx 2.459 \times 10^{-3}$
- ...
- $\eta_{v_2v_{31}} = \frac{1}{515.57} \approx 1.939 \times 10^{-3}$

For the first iteration of the solution, the initial pheromone value $\tau_{ij} = 1$ which will be updated after each iteration.

$$P_{v_2v_4} = \frac{(\tau^1_{v_2v_4})(\eta^3_{v_2v_4})}{\sum(\tau^1_{v_2v_4})(\eta^3_{v_2v_4})} = \frac{(1)^1 \cdot (5.386 \times 10^{-3})^3}{3.1045 \times 10^{-7}} \approx 0.5034$$

$$P_{v_2v_6} = \frac{(\tau^1_{v_2v_6})(\eta^3_{v_2v_6})}{\sum(\tau^1_{v_2v_6})(\eta^3_{v_2v_6})} = \frac{(1)^1 \cdot (3.297 \times 10^{-3})^3}{3.1045 \times 10^{-7}} \approx 0.1155$$

$$P_{v_2v_7} = \frac{(\tau^1_{v_2v_7})(\eta^3_{v_2v_7})}{\sum(\tau^1_{v_2v_7})(\eta^3_{v_2v_7})} = \frac{(1)^1 \cdot (2.599 \times 10^{-3})^3}{3.1045 \times 10^{-7}} \approx 0.0565$$

$\vdots$

$$P_{v_2v_{31}} = \frac{(\tau^1_{v_2v_{31}})(\eta^3_{v_2v_{31}})}{\sum(\tau^1_{v_2v_{31}})(\eta^3_{v_2v_{31}})} = \frac{(1)^1 \cdot (1.929 \times 10^{-3})^3}{3.1045 \times 10^{-7}} \approx 0.0235$$

From these values, the most likely pairing to be selected in the first iteration is between vertices v_2 and v_4, with a probability of 50.34%.

For updating pheromone value, assuming $Q = 100$ and $m = 3$ ants with the following constructed paths:

For m = 1, $v_2 \to v_6 \to v_4 \to v_2$ with a total length $L = 303.24 + 488.88 + 185.64 = 977.76$.

For m = 2, $v_2 \to v_4 \to v_2$ with a total length $L = 185.64 + 185.64 = 371.28$.

For m = 3, $v_2 \to v_6 \to v_7 \to v_2$ with a total length $L = 303.24 + 81.51 + 384.75 = 769.50$.

Focusing on the pheromone value of $\tau_{v_2v_4}$ for clear illustration and same process applies to all pheromone value. Using Equation 4, the contributions are:

$\Delta\tau^1_{v_2v_4} = \frac{100}{977.76} \approx 0.102$.

$\Delta\tau^2_{v_2v_4} = \frac{100}{371.38} \approx 0.269$

$\Delta\tau^3_{v_2v_4} = \frac{100}{769.50} = 0$, since edge (v_2, v_4) is not covered.

Then, using Equation 3,

$$\tau_{v_2v_4} = (1 - 0.5)(1) + 0.102 + 0.269 + 0 = 0.871$$

The updated pheromone value of $\tau_{v_2v_4}$ is 0.871.

This process is repeated for several iterations, and the resulting pairing for this case is between vertices v_2 and v_4 pair. The same procedure is applied to all odd-degree vertices to be paired until a complete set of pairings are formed.

Ant Colony Optimization Generation Output

Since the Ant Colony Optimization is a Probabilistic Metaheuristic Algorithm Model, there will be differences when running the algorithm and getting different vertex pairings per generation. From the Ten Generations ran by the Algorithm, the following outputs presented at Table 4 shows the lowest total distance sum which will be used as the near-optimal solution.

Table 4

Vertex Pairings from Ten Generations

Pair	Gen 1	Gen 2	Gen 3	Gen 4	Gen 5	Gen 6	Gen 7	Gen 8	Gen 9	Gen 10
1	(2, 4)	(2, 4)	(2, 4)	(2, 4)	(2, 4)	(2, 4)	(2, 4)	(2, 4)	(2, 6)	(2, 4)
2	(6, 8)	(6, 8)	(6, 7)	(6, 8)	(6, 7)	(6, 8)	(6, 7)	(6, 30)	(4, 11)	(6, 8)
3	(7, 9)	(7, 9)	(8, 9)	(7, 9)	(8, 11)	(7, 9)	(8, 9)	(7, 8)	(7, 8)	(7, 9)
4	(10, 11)	(10, 11)	(10, 11)	(10, 11)	(9, 10)	(10, 11)	(10, 11)	(9, 10)	(9, 24)	(10, 11)
5	(12, 13)	(12, 13)	(12, 13)	(12, 13)	(12, 13)	(12, 13)	(12, 13)	(11, 12)	(10, 13)	(12, 13)
6	(15, 24)	(15, 17)	(15, 19)	(15, 17)	(15, 17)	(15, 20)	(15, 17)	(13, 15)	(12, 15)	(15, 17)
7	(17, 21)	(18, 19)	(17, 27)	(18, 19)	(18, 19)	(17, 19)	(18, 19)	(17, 23)	(17, 19)	(18, 19)
8	(18, 19)	(20, 21)	(18, 20)	(20, 21)	(20, 21)	(18, 21)	(20, 21)	(18, 19)	(18, 20)	(20, 21)
9	(20, 22)	(22, 23)	(21, 22)	(22, 23)	(22, 23)	(22, 23)	(22, 23)	(20, 21)	(21, 22)	(22, 23)
10	(23, 29)	(24, 31)	(23, 24)	(24, 31)	(24, 31)	(24, 31)	(24, 31)	(22, 24)	(23, 31)	(22, 23)
11	(27, 28)	(27, 29)	(28, 29)	(27, 29)	(27, 29)	(27, 29)	(27, 29)	(27, 29)	(27, 29)	(24, 31)
12	(30, 31)	(28, 30)	(30, 31)	(28, 30)	(28, 30)	(28, 30)	(28, 30)	(28, 31)	(28, 30)	(27, 29)
Total	**2047**	**1263**	**2016**	**1263**	**1399**	**1972**	**1219**	**1877**	**2606**	**1263**

Among the generated vertex pairings from the Ten Generations, the least total distance among them has 1219 meters from the 7th Generation from the edges of the newly paired vertices. Therefore, the near-optimal solution to be presented are as follows

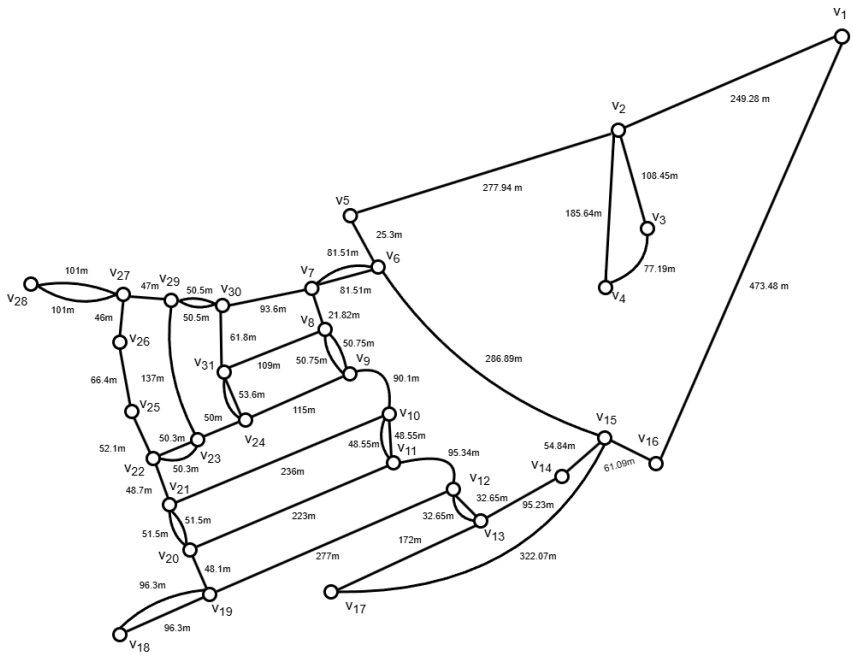
$$Pair = \{(2,4),(6,7),(8,9),(10,11),(12,13),(15,17),$$
$$(18,19),(20,21),(22,23),(24,31),(27,28),(29,30)\}$$

These pairings minimize the total additional distance required to duplicate edges and transform the graph into an Eulerian graph.

Traversing the Eulerian Map

Figure 3

Eulerian Graph of Barangay Virgen Delas Flores



After the odd-degree pairing of vertices, Figure 3 shows the completed Eulerian Graph; because the graph does not contain any more odd-degree vertices in which case an Eulerian Circuit is possible, solving the Chinese Postman Problem. The Graph shown at Figure 4 depicts the graph traversal routing for the eulerian circuit. Table 5 depicts the Eulerian Tour of the Chinese Postman Problem as well as its cost terms and their sum.

The optimality of the resulting route is dependent on the assigned edge weights in which it represents the distance between vertices. Any changes to these weights such as included constraints such as road conditions, schedule time, and other constraints that may affect the weights then it can affect the computed shortest paths in which may also affect the vertex pairings of the odd-degree vertices.

Since the Ant Colony Optimization (ACO) relies on the distance value's shortest path values, variations in edge weights leads to different pairing combinations and

Figure 4

Eulerian Tour of Barangay Virgen Delas Flores

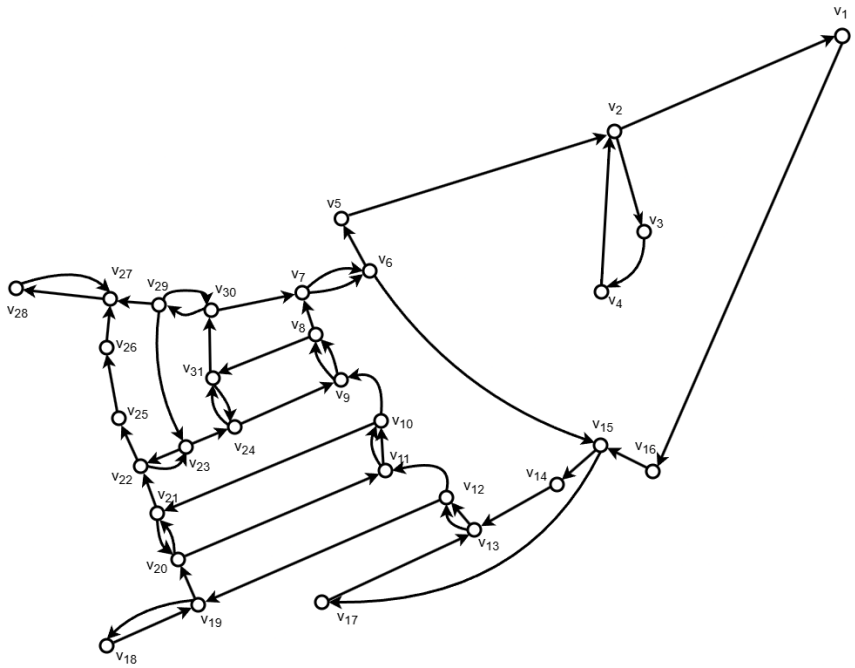


Table 5

Eulerian Tour Generated by ACO

Euler Tour Path	Weight (in meters)	Total
$v_{15} \to v_{14} \to v_{13} \to v_{12} \to$ $v_{11} \to v_{10} \to v_9 \to v_{24} \to$ $v_{31} \to v_{24} \to v_{23} \to v_{29} \to$ $v_{27} \to v_{28} \to v_{27} \to v_{26} \to$ $v_{25} \to v_{22} \to v_{23} \to v_{22} \to$ $v_{21} \to v_{10} \to v_{11} \to v_{20} \to$ $v_{21} \to v_{20} \to v_{19} \to v_{18} \to$ $v_{19} \to v_{12} \to v_{13} \to v_{17} \to$ $v_{15} \to v_{16} \to v_1 \to v_2 \to$ $v_4 \to v_3 \to v_2 \to v_5 \to$ $v_6 \to v_7 \to v_8 \to v_{31} \to$ $v_{30} \to v_{29} \to v_{30} \to v_7 \to$ $v_6 \to v_{15}$	$54.84 + 95.23 + 32.65 + 95.34 + 48.55 + 90.10 +$ $50.75 + 50.75 + 115.00 + 53.60 + 53.60 + 50.00 +$ $137.00 + 47.00 + 101.00 + 101.00 + 46.00 + 66.40 +$ $52.10 + 50.30 + 50.30 + 48.70 + 236.00 + 48.55 +$ $223.00 + 51.50 + 51.50 + 48.10 + 96.30 + 96.30 +$ $277.00 + 32.65 + 172.00 + 322.07 + 61.09 + 473.48$ $+ 249.28 + 185.64 + 77.19 + 108.45 + 277.94 + 25.30$ $+ 81.51 + 21.82 + 109.00 + 61.80 + 50.50 + 50.50 +$ $93.60 + 81.51 + 286.89$	5440.68

alternative Eulerian routes. However, due to the probabilistic nature of the algorithm, it remains robust in exploring multiple candidate solutions and converging toward near-optimal pairings even when the data changes.

This indicates that while the exact route may vary under different conditions,

the model used remains adaptable in different scenarios. The model can therefore accommodate updated data inputs, making it suitable for dynamic or evolving real-world environments where road conditions and operational constraints may change over time.

SUMMARY AND CONCLUSION

Summary of Findings

The study applies Graph Theory to optimize garbage collection routes in Barangay Virgen Delas Flores. The Barangay's route framework was modeled as a weighted graph, with intersections as vertices and roads as edges weighted by distance. For analysis, it shows that several vertices had odd-degree vertices that indicates that the network is a non-Eulerian graph, which means it is impossible to traverse the entire network without repeating some edges. To resolve this, the Chinese Postman Problem will be applied to ensure that all roads can still be traversed while minimizing repeated odd-degree vertices. At the same time, the Dijkstra's algorithm was used to compute the shortest paths between them.

Mathematical modeling was also applied to optimize garbage collection routes in the barangay. Due to the size of the odd-degree vertices being 24 total vertices, there would be approximately 316,143,225 distinct pairings which would become computationally exhaustive. Hence why the Ant Colony Optimization (ACO) was utilized as a meta-heuristic approach to determine efficient pairings. This approach minimized repeated paths and reduced the total travel distance while handling the difficulty of the routing problem.

Conclusions

By utilizing both Graph Theory and Mathematical Modeling, it provides a systematic and practical frame for optimizing garbage collection routes. By the use of Chinese Postman Problem and Dijkstra's Algorithm, it ensures that all roads are covered for garbage collection with the utilization of Ant Colony Optimization as a meta-heuristic method to determine efficient pairings. Combining both demonstrates how mathematical concepts can be applied to real-world problems and also serve as a practical guide for barangays in optimizing their garbage collection waste efficiently.

The results demonstrate that systematic optimization significantly reduces redundant traversal compared to non-optimized routing, while ensuring complete coverage of all given vertices. Moreover, the study highlights the effectiveness of the Ant Colony Optimization (ACO) in handling large-scale pairing problems where exact methods become computationally impractical.

Overall, the study provides a mathematically grounded and operationally applicable framework for improving garbage collection efficiency, which can be adapted for similar routing problems in other localities.

REFERENCES

Black, P. (2024). *Dijkstra's algorithm.* National Institute of Standards and Technology. https://xlinux.nist.gov/dads/HTML/dijkstraalgo.html

Calverd, N. (2025, June). *Eulerian & semi-eulerian graphs.* Save My Exams. https://www.savemyexams.com/a-level/further-maths/edexcel/17/decision-1/revision-notes/algorithms-and-graph-theory/graphs/eulerian-and-semi-eulerian-graphs/

Carlson, S. (2026, February). *Graph theory.* Encyclopaedia Britannica. https://www.britannica.com/topic/graph-theory

Navone, E. C. (2020, September). *Dijkstra's shortest path algorithm - a detailed and visual introduction.* FreeCodeCamp. https://www.freecodecamp.org/news/dijkstras-shortest-path-algorithm-visual-introduction/

Reno, J. (2015). Waste and waste management. *Annual Review of Anthropology*, *44*(1), 557–572. https://doi.org/10.1146/annurev-anthro-102214-014146

Roopawalla, H. (2024, April). *Ant colony optimization.* Medium. https://medium.com/@hasnain.roopawalla/ant-colony-optimization-1bbc346c2da5

Sgarro, G. A., & Grilli, L. (2023). Ant colony optimization for chinese postman problem. *Neural Computing and Applications*, *36*(6), 2901–2920. https://doi.org/10.1007/s00521-023-09195-4

Siddique, N. H., & Adeli, H. (2017). *Nature-inspired computing: Physics and chemistry-based algorithms.* Chapman; Hall/CRC. https://doi.org/10.1201/9781315118628

Wang, F., Peng, X., Wei, R., Qin, Y., & Zhu, X. (2018). Environmental behavior research in resources conservation and management: A case study of resources, conservation and recycling. *Resources, Conservation and Recycling*, *141*, 431–440. https://doi.org/10.1016/j.resconrec.2018.10.024

APPENDIX A

Vertex and Edge List

Barangay Virgen Delas Flores depicts many Intersections, Dead-Ends, and potential garbage collection points. As mentioned, roads only accessible by the garbage truck are to be included for the graph. Table 6 shows the detailed locations and information of the vertices presented while Table 7 shows the full edge list of the graph.

APPENDIX B

SageMath/Python Codes

The following Python and SageMath codes are used to process the graph integration to the Ant Colony Optimization Model. The Python code integrates SageMath so some functions such as graph figures will be included.

The following code is the initialization of the Graph to be used for the metaheuristic model:

```
G = Graph([(1,2,249),(2,3,108),(3,4,77),(2,5,278),(1,16,474),(5,6,25)
    ,(6,15,287),(15,16,61),(6,7,82),(7,8,22),(7,30,94),(30,29,51)
    ,(29,27,47),(28,27,101),(27,26,46),(26,25,66),(25,22,52),(29,23,137)
    ,(30,31,62),(31,24,54),(8,9,51),(31,8,109),(24,9,115),(9,10,90)
    ,(22,21,49),(21,10,236),(21,20,52),(10,11,49),(20,11,223),(20,19,48)
    ,(11,12,95),(12,13,33),(19,12,277),(18,19,96),(17,13,172),(13,14,95)
    ,(14,15,54),(22,23,50),(24,23,50)])

G.show(edge_labels=True, figsize=14)

```

The following code converts the graph into an adjacency matrix which becomes a 2D List. It then takes all the odd-degree vertices to be used for the vertex pairings and stores them into an array:

```
Adj = G.adjacency_matrix()
Adj.str()

import numpy as np
Adj = np.array(Adj)
degrees = Adj.sum(axis=1)
vertex = 0
odd_vertices = []
for i in degrees:
    vertex += 1
    if i % 2 == 1:
        odd_vertices.append(vertex) # Adds that Vertex to the array if
    odd

odd_vertices = np.array(odd_vertices)

```

The following code starts the algorithmic process of the Ant Colony Optimization by first defining the Global variables to be used as values to the algorithm and making a

copy of the odd vertices array:

```
# Global Variables
alpha = 1
beta = 5
rho = 0.1
Q = 1
Tao = {}

# Initialixing Tao Matrix (2D Array)
for u in odd_vertices:
    for v in odd_vertices:
        if u != v:
            Tao[(u,v)] = 1

Best_Distance = 0
Best_Pairs = []
remaining_odds = odd_vertices

```

The following code shows how the value where the Ant Colony Optimization's Probability of choosing the next vertex pairings:

```
def normalize():
    p = []
    i = remaining_odds[0]

    denom = 0
    for j in remaining_odds:
        if j == i:
            continue
        d = G.shortest_path_length(i, j, by_weight=True)
        denom += (Tao[(i,j)]**alpha) * (1/d)**beta

    for j in remaining_odds:
        if j == i:
            p.append(0)
            continue

        d = G.shortest_path_length(i, j, by_weight=True)
        num = (Tao[(i,j)]**alpha) * (1/d)**beta
        p.append(num / denom)

    return p
```

The following code is the ant choosing the next vertex by using the probability values.

```
import random

def choose():
    probabilities = normalize()
    i = remaining_odds[0]
```

```

    r = random.random()
    cumulative = 0

    for j, p in enumerate(probabilities):
        cumulative += p
        if r <= cumulative:
            return i, remaining_odds[j]
```

The following code shows the appending of the found vertex pairs into an array

```
def pairing():
global remaining_odds
pairs = []

while len(remaining_odds) > 1:
    x, y = choose()
    pairs.append((x,y))
    new_remaining_odds = []
    for v in remaining_odds:
        if v != x and v != y:
            new_remaining_odds.append(v)
    remaining_odds = new_remaining_odds

return pairs
```

The following code shows the Pheromone Update of the entire algorithm for the next vertex pairing:

```
def update_pheromones(pairs):
    global Tao

    # Evaporation
    for edge in Tao:
        Tao[edge] *= (1 - rho)

    # Reinforcement
    for (u, v) in pairs:
        dist = G.shortest_path_length(u, v, by_weight=True)
        Tao[(u,v)] += Q / dist
    Tao[(v,u)] += Q / dist
```

The following code runs the generations of the entire ant colony optimization algorithmand prints the vertex pairings:

```
for generation in range(10):
    remaining_odds = list(odd_vertices)

    pairs = pheromone()

    total_dist = 0
```

```python

    for u, v in pairs:
        dist = G.shortest_path_length(u, v, by_weight=True)
        total_dist += dist

    update_pheromones(pairs)

    print(f"Generation {generation+1} | Total Distance: {total_dist}")
    for i in pairs:
        print(f"Pairs found: {i}")
```

The following code implements the new vertex pairings and integrates it into a new duplicated graph, making the Non-Eulerian Graph into an Eulerian Graph. Best_Pairs was manually chosen for the process.

```python
I = Graph(G, multiedges=True)

for i in Best_Pairs:
    print(i[0], i[1])
    cost = G.shortest_path_length(i[0], i[1], by_weight=True)
    I.add_edge(i[0],i[1],cost)
```

Table 6

Full Vertex List

Vertex ID	Description	Type	Lat	Long
v_1	Gil Carlos-Aldama St. Intersection	Intersection	14.9501	120.8924
v_2	Gil Carlos-E Delos Angeles St. Intersection	Intersection	14.9492	120.8903
v_3	E Delos Angeles St.	Road	14.9483	120.8906
v_4	E Delos Angeles St.	Dead End	14.9479	120.8903
v_5	E Padre Damaso-Gil Carlos St. Intersection	Intersection	14.9483	120.8851
v_6	E Padre Damaso St.	Intersection	14.9480	120.8881
v_7	Dau St.	Intersection	14.947	120.8875
v_8	Dau St.	Intersection	14.9474	120.8876
v_9	Mulawin St.	Intersection	14.9470	120.8878
v_10	Molave St.	Intersection	14.9467	120.8882
v_11	Molave St.	Intersection	14.9462	120.8882
v_12	Molave St.	Intersection	14.9460	120.8889
v_13	Alido Townhouse	Intersection	14.9457	120.8890
v_14	Padre Damaso St.	Road	14.9461	120.8898
v_15	Padre Damaso St.	Intersection	14.9465	120.8902
v_16	Aldama St.	Road	14.9462	120.8907
v_17	Alido Townhouse	Endpoint	14.9450	120.8876
v_18	Loyong St.	Endpoint	14.9447	120.8857
v_19	Loyong St.	Road	14.9450	120.8865
v_20	Loyong St.	Road	14.9454	120.8863
v_21	Molave St.	Road	14.9458	120.8861
v_22	Mulawin St.	Road	14.9463	120.8860
v_23	Tanguile St.	Intersection	14.9464	120.8864
v_{24}	Mulawin-Narra St. Intersection	Intersection	14.9466	120.8868
v_{25}	Loyong St.	Road	14.9467	120.8858
v_{26}	Loyong St.	Road	14.9473	120.8857
v_{27}	Loyong-Dayau St.	Intersection	14.9477	120.8857
v_{28}	Loyong	Dead End	14.9478	120.8848
v_{29}	Dau-Tanguile St.	Intersection	14.9477	120.8861
v_{30}	Dau-Narra St.	Intersection	14.9477	120.8866
v_{31}	Narra St.	Intersection	14.9471	120.8866
v_{50}	Bocaue Toll Plaza	Intersection	14.8024	120.9424

Table 7

Full Edge List During Peak Hours

Edge ID	From	To	Road Type	Distance (in meters)
e_1	v_1	v_2	Municipal Road	249.28
e_2	v_2	v_3	Municipal Road	108.45
e_3	v_3	v_4	Barangay Road	77.19
e_4	v_2	v_5	Barangay Road	277.94
e_5	v_5	v_6	Barangay Road	25.3
e_6	v_6	v_7	Barangay Road	81.51
e_7	v_7	v_8	Barangay Road	21.82
e_8	v_8	v_9	Barangay Road	50.75
e_9	v_9	v_{10}	Barangay Road	90.1
e_{10}	v_{10}	v_{11}	Barangay Road	48.55
e_{11}	v_{11}	v_{12}	Barangay Road	95.34
e_{12}	v_{12}	v_{13}	Barangay Road	32.65
e_{13}	v_{13}	v_{14}	Barangay Road	95.23
e_{14}	v_{14}	v_{15}	Barangay Road	54.84
e_{15}	v_{15}	v_{16}	Barangay Road	61.09
e_{16}	v_1	v_{16}	Municipal Road	473.48
e_{17}	v_6	v_{15}	Barangay Road	286.89
e_{18}	v_7	v_{30}	Barangay Road	93.6
e_{19}	v_{30}	v_{29}	Barangay Road	50.5
e_{20}	v_{29}	v_{27}	Barangay Road	47
e_{21}	v_{27}	v_{28}	Barangay Road	101
e_{22}	v_{27}	v_{26}	Barangay Road	46
e_{23}	v_{26}	v_{25}	Barangay Road	66.4
e_{24}	v_{25}	v_{22}	Barangay Road	52.1
e_{25}	v_{22}	v_{21}	Barangay Road	48.7
e_{26}	v_{21}	v_{20}	Barangay Road	51.5
e_{27}	v_{20}	v_{19}	Barangay Road	48.1
e_{28}	v_{19}	v_{18}	Barangay Road	96.3
e_{29}	v_{21}	v_{10}	Municipal Road	236
e_{30}	v_{20}	v_{11}	Municipal Road	223
e_{31}	v_{19}	v_{12}	Municipal Road	277
e_{32}	v_{23}	v_{22}	Barangay Road	50.3
e_{33}	v_{24}	v_{23}	Barangay Road	50
e_{34}	v_{31}	v_8	Municipal Road	109
e_{35}	v_{24}	v_9	Municipal Road	115
e_{36}	v_{17}	v_{13}	Municipal Road	172
e_{37}	v_{30}	v_{31}	Barangay Road	61.8
e_{38}	v_{31}	v_{24}	Barangay Road	53.6